

Chapitre 5: Gestion de la pile et du Tas

Cours 8: Gestion de la pile

Plan

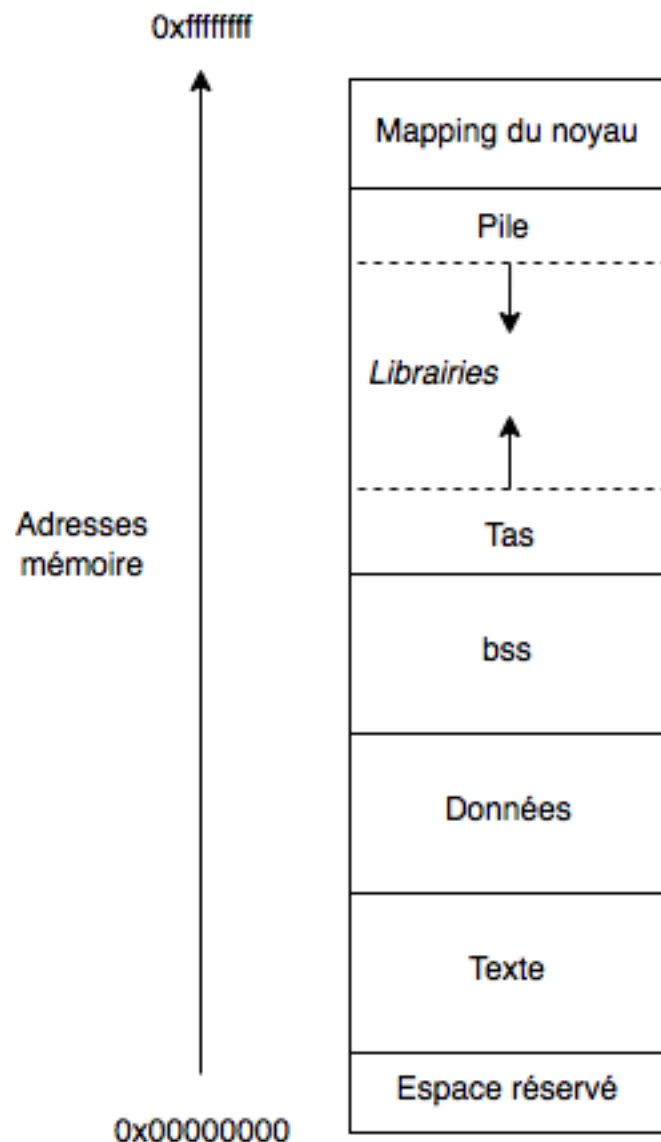
- 1. Qu'est-ce que la pile ?**
- 2. Qu'est ce qu'une stack frame**
- 3. Appel d'une fonction: ce qui se passe**
- 4. Récursion et risques**
- 5. Introduction au buffer overflow**
- 6. Exemple reel de code vulnerable**
- 7. Principe d'exploitation**
- 8. Protections modernes**
- 9. Conclusion**

Plan

- 1. Qu'est-ce que la pile ?**
2. Qu'est ce qu'une stack frame
3. Appel d'une fonction: ce qui se passe
4. Récursion et risques
5. Introduction au buffer overflow
6. Exemple reel de code vulnerable
7. Principe d'exploitation
8. Protections modernes
9. Conclusion

Qu'est ce que la pile ?

- La **pile (stack)** est une zone mémoire utilisée pour gérer :
 - Les **variables locales** des fonctions
 - Les **adresses de retour** lors des appels de fonction
 - Les **paramètres passés à une fonction**
- Elle fonctionne selon le principe **LIFO (Last In, First Out)**: le dernier élément ajouté est le premier retiré (exemple. pile d'assiette).
- Contrairement à la pile d'assiette, la pile empile ses éléments vers le bas. Donc ce qu'on appelle le haut de la pile, c'est finalement l'adresse la plus basse de la pile. Plus on empile des valeurs dans la pile, plus les adresses diminuent.



Plan

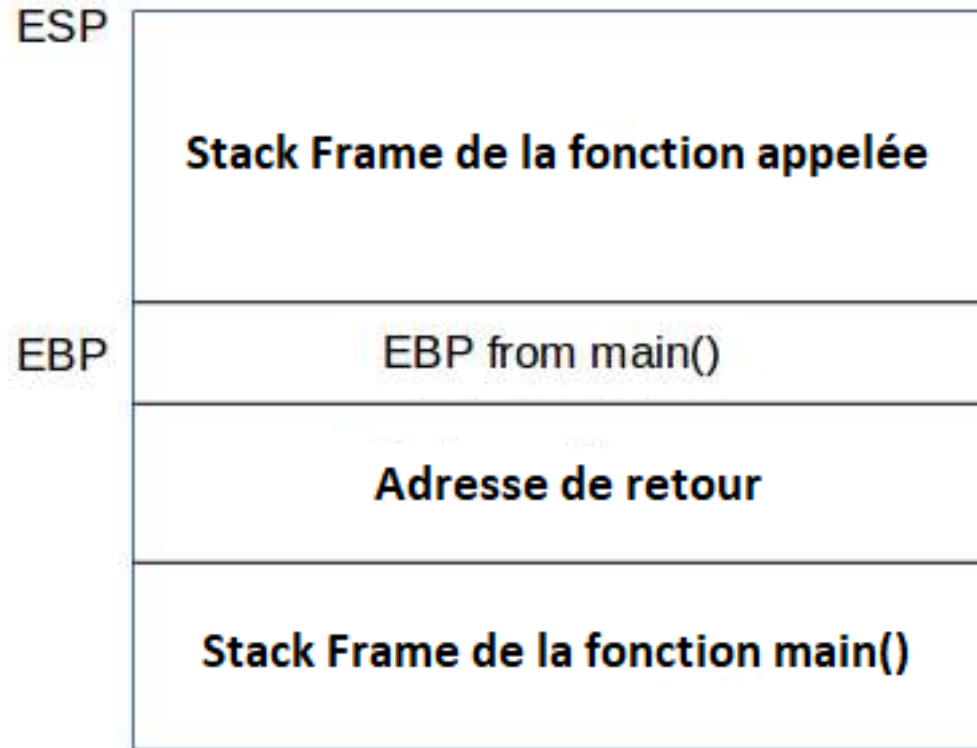
1. Qu'est-ce que la pile ?
- 2. Qu'est ce qu'une stack frame**
3. Appel d'une fonction: ce qui se passe
4. Récursion et risques
5. Introduction au buffer overflow
6. Exemple reel de code vulnerable
7. Principe d'exploitation
8. Protections modernes
9. Conclusion

Qu'est ce qu'une stack frame ?

- Cette structure LIFO est extrêmement utile. En effet, lors de l'appel d'une fonction, toutes les données nécessaires pour l'exécution de la fonction, ainsi que pour le retour à l'état initial sont empilées dans une zone de la pile qu'on appelle **Stack Frame** ou **frame** brièvement.
- Une fois la fonction terminée, il faut donc retourner à la ligne suivant son appel, et ceci se fait en dépilant tout ce qui a été précédemment empilé, laissant intact le reste de la pile et les autres éventuelles frames.

Qu'est ce qu'une stack frame ?

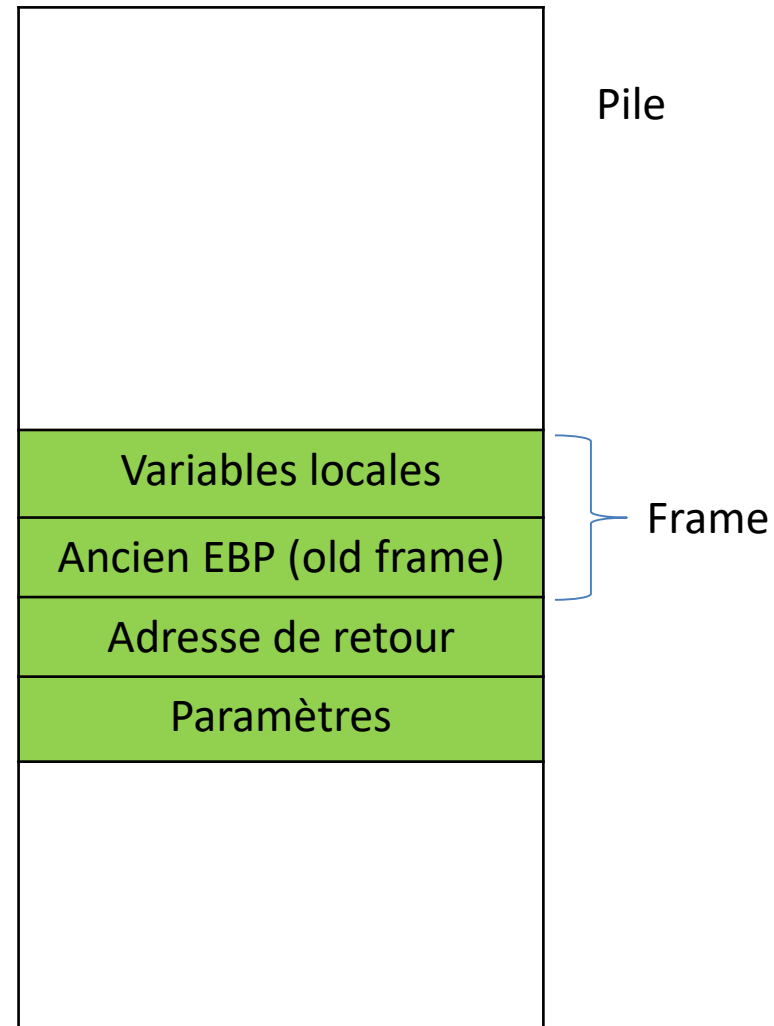
- Lors de l'appel d'une fonction, le SE crée une frame dans la pile et enregistre dedans le contexte d'exécution ainsi que les variables passées à la fonction.
- Le registre EBP garde en mémoire l'adresse du début de la frame.
- Le registre ESP garde en mémoire l'adresse du haut de la pile (donc l'adresse la plus basse). Il est donc mis à jour à chaque modification de la pile (ajout d'une valeur ou suppression de la dernière valeur).
- Ainsi, la frame courante se situe entre l'adresse contenue dans EBP et l'adresse contenue dans ESP.



Qu'est ce qu'une stack frame ?

Donc, la pile est organisée en **frames**. Lorsqu'on appelle une fonction, le SE crée une frame et empile les informations suivantes en commençant par:

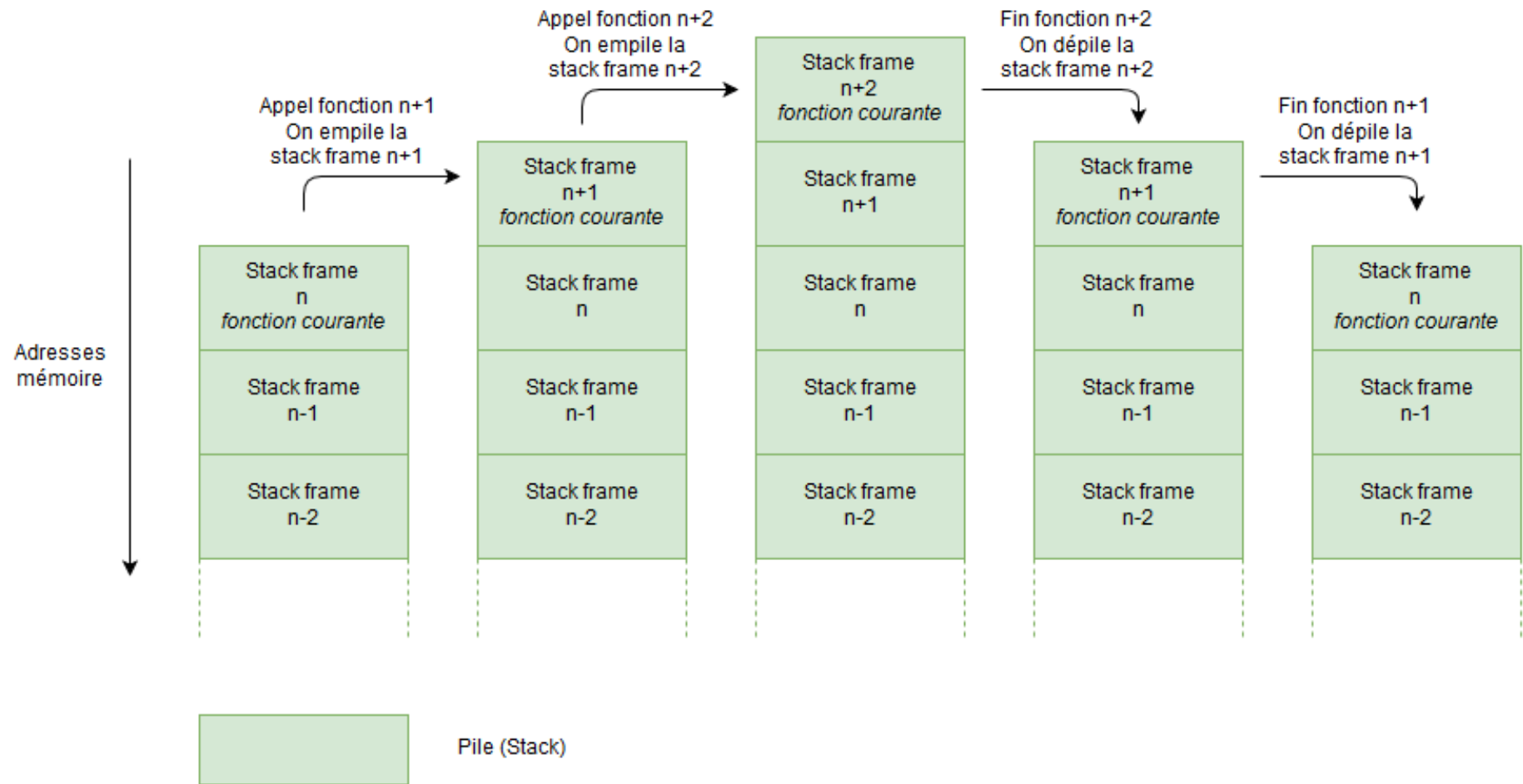
1. Les paramètres de la fonction qui seront en bas.
2. L'adresse de retour (où reprendre l'exécution après l'appel).
3. L'ancien EBP permet de revenir au frame précédent
4. Les variables locales de la fonction en haut de la frame.
5. À la fin, la fonction détruit son frame et la pile revient à l'état précédent.



Plan

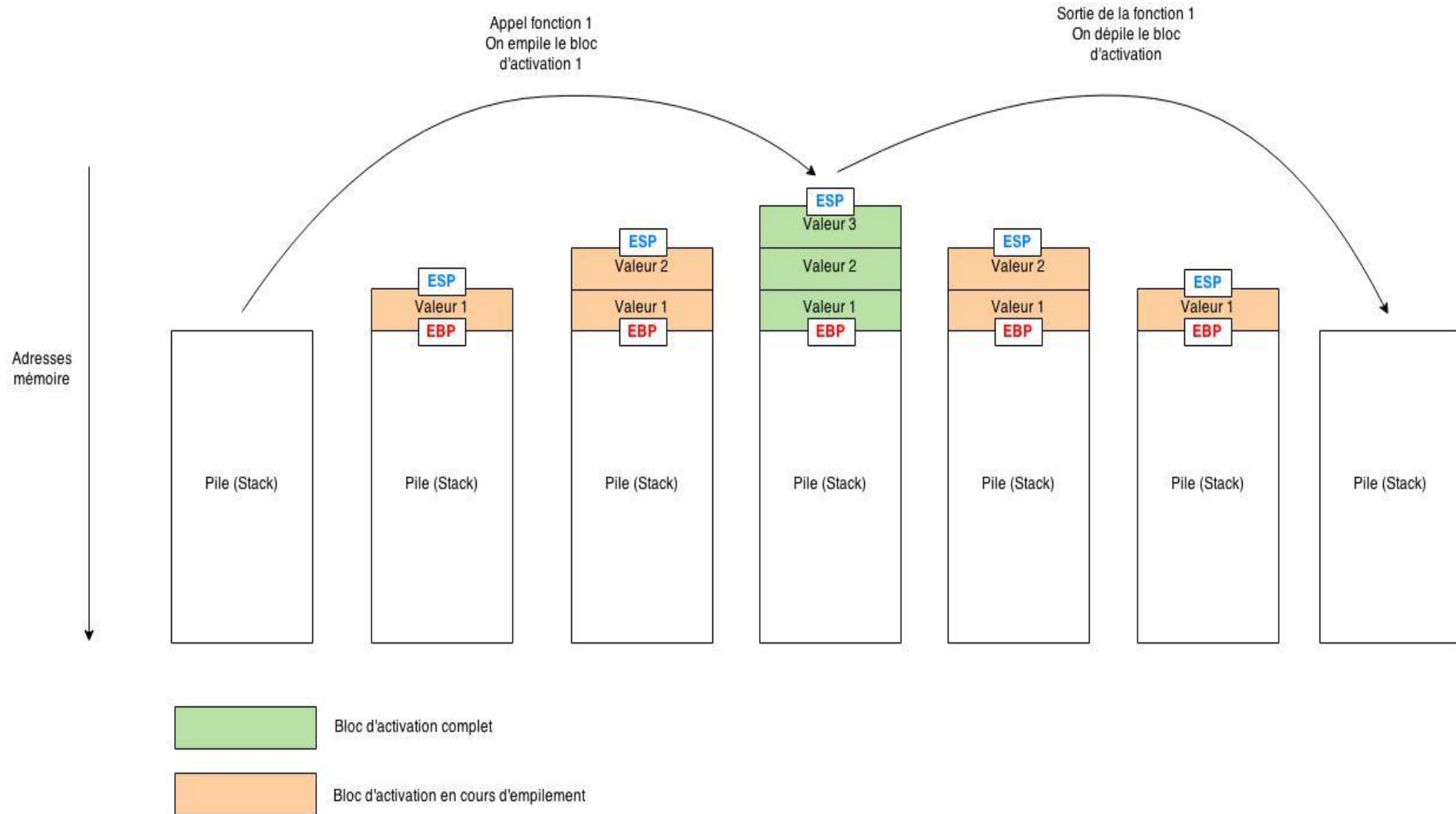
1. Qu'est-ce que la pile ?
2. Qu'est ce qu'une stack frame
- 3. Appel d'une fonction: ce qui se passe**
4. Récursion et risques
5. Introduction au buffer overflow
6. Exemple reel de code vulnerable
7. Principe d'exploitation
8. Protections modernes
9. Conclusion

Appel d'une fonction: ce qui se passe



Appel d'une fonction: ce qui se passe

Voici un schéma qui illustre le rôle des registres EBP et ESP :

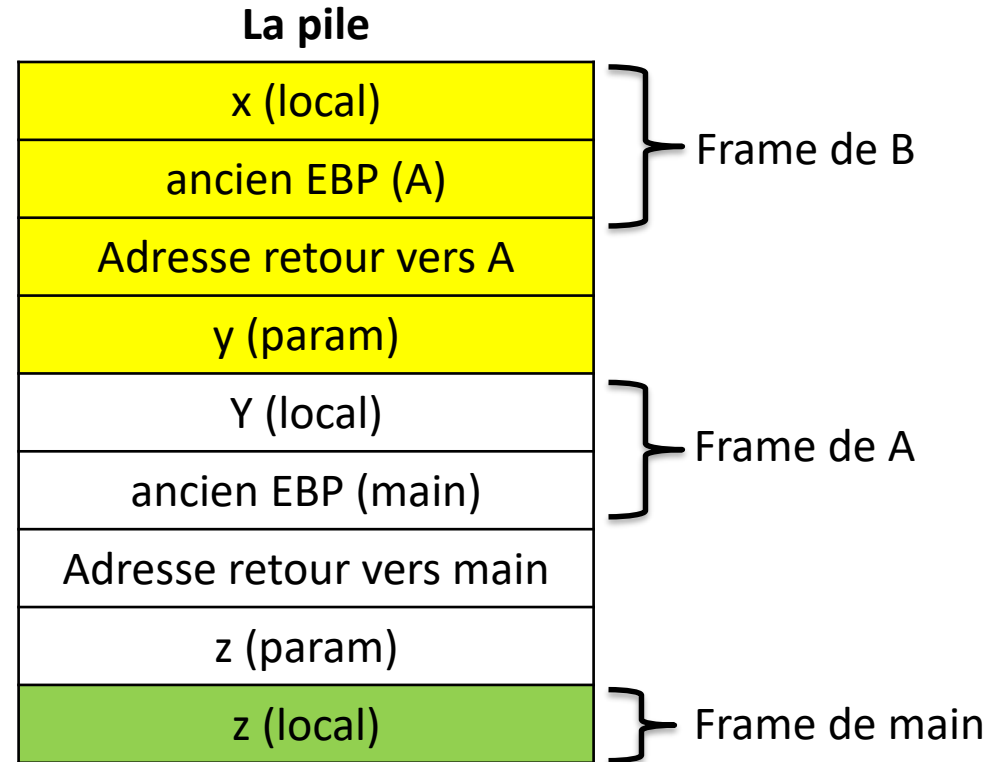


Appel d'une fonction: ce qui se passe

```
void B(int b) {  
    int x = b + 1;  
    printf("x = %d\n", x);  
}
```

```
void A(int a) {  
    int y = a * 2;  
    B(y);  
}
```

```
int main() {  
    int z = 10;  
    A(z);  
}
```



Ordre d'appels : $\text{main} \rightarrow \text{A} \rightarrow \text{B}$

À chaque fonction :

- un nouveau frame est créé
- puis détruit à la fin

Appel d'une fonction: ce qui se passe

Exercice:

Que renvoie cette fonction après son appel ?

```
int* danger() {  
    int x = 5;  
    return &x;  
}
```

Réponse:

x sera détruit → pointeur mort

Les **variables locales** sont détruites à la sortie de la fonction.

Plan

1. Qu'est-ce que la pile ?
2. Qu'est ce qu'une stack frame
3. Appel d'une fonction: ce qui se passe
- 4. Récursion et risques**
5. Introduction au buffer overflow
6. Exemple reel de code vulnerable
7. Principe d'exploitation
8. Protections modernes
9. Conclusion

Réursion et Risques

- Chaque appel récursif crée un **nouveau frame sur la pile**
- La pile peut **déborder** si :
 - Il y a trop d'appels récursifs (stack overflow), ou aussi
 - Trop de variables locales volumineuses

Exemple :

```
int factoriel(int n) {  
    if (n==0)  
        return 1;  
    return n*factorie(n-1);  
}
```

Plan

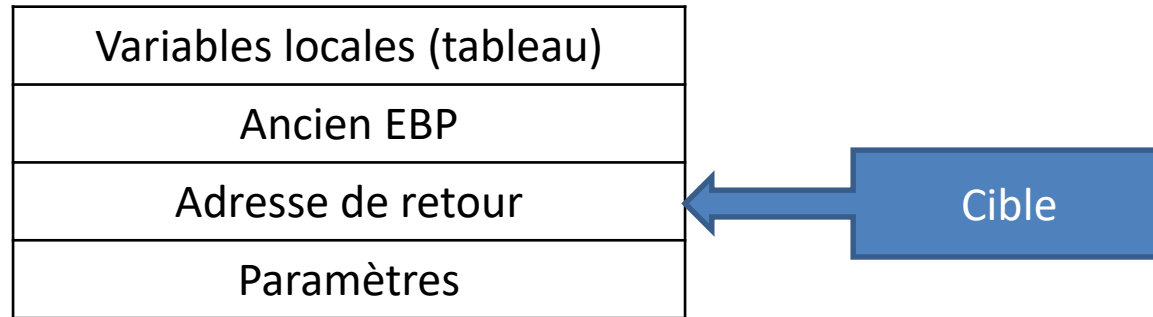
1. Qu'est-ce que la pile ?
2. Qu'est ce qu'une stack frame
3. Appel d'une fonction: ce qui se passe
4. Récursion et risques
- 5. Introduction au buffer overflow**
6. Exemple reel de code vulnerable
7. Principe d'exploitation
8. Protections modernes
9. Conclusion

Introduction au Buffer Overflow

- Un **buffer overflow** (dépassement de tampon) se produit lorsqu'un programme écrit **plus de données que la taille prévue** dans un tableau, souvent un `char[]`.
- Comme les variables locales sont stockées **sur la pile**, un dépassement peut écraser :
 - D'autres variables locales
 - **L'adresse de retour (Dangereux)**

Introduction au Buffer Overflow

Pourquoi la frame est vulnérable ?



Si un tableau local déborde, il peut atteindre l'adresse de retour et la modifier.

Plan

1. Qu'est-ce que la pile ?
2. Qu'est ce qu'une stack frame
3. Appel d'une fonction: ce qui se passe
4. Récursion et risques
5. Introduction au buffer overflow
- 6. Exemple reel de code vulnerable**
7. Principe d'exploitation
8. Protections modernes
9. Conclusion

Exemple réel de code vulnérable

```
void vuln() {  
    char buffer[10];  
    printf("Entrez du texte : ");  
    gets(buffer); // DANGEREUX  
    printf("%s\n", buffer);  
}
```

Saisie de l'attaquant :

AAAAAAAAAAAAAAAAAAAAAAAAAAAA

Écrase :

- bufffer
- ancien EBP
- adresse de retour

Pourquoi ?

Parce que `gets()` (ou `scanf("%s")` sans limite) permet d'écrire **plus de 10 caractères**, écrasant la pile.

Exemple réel de code vulnérable

Mémoire attendue :

[AAAA AAAA AA]

[ancien EBP]

[adresse de retour]

Mémoire après overflow:

AAAA AAAA AA

AAAAAAAAAAAA

AAAAAAAAAAAA ← Adresse de retour écrasée

Plan

1. Qu'est-ce que la pile ?
2. Qu'est ce qu'une stack frame
3. Appel d'une fonction: ce qui se passe
4. Récursion et risques
5. Introduction au buffer overflow
6. Exemple reel de code vulnerable
- 7. Principe d'exploitation**
8. Protections modernes
9. Conclusion

Principe d'exploitation

L'attaquant peut :

1. Écrire trop de caractères pour atteindre l'adresse de retour
2. Remplacer cette adresse par une **adresse vers un code injecté**

Exemple symbolique :

Entrée de l'attaquant :

[remplissage][adresse_malicious]

Objectif:

→ Faire exécuter du code arbitraire lors du ret.

Plan

1. Qu'est-ce que la pile ?
2. Qu'est ce qu'une stack frame
3. Appel d'une fonction: ce qui se passe
4. Récursion et risques
5. Introduction au buffer overflow
6. Exemple reel de code vulnerable
7. Principe d'exploitation
- 8. Protections modernes**
9. Conclusion

Protection modernes

Les systèmes modernes utilisent plusieurs mécanismes pour empêcher les exploitations :

1. **DEP / NX-bit**

- Empêche l'exécution du code dans la pile
- La pile devient "non exécutable"

2. **ASLR (Address Space Layout Randomization)**

- Les adresses mémoire sont aléatoires, donc difficiles à prédire

3. **Stack Canaries**

- Valeurs secrètes placées entre variables locales et adresse de retour
- Si elles sont modifiées → crash automatique

4. **Fonctions sécurisées**

- `gets()` supprimée du standard
- Utiliser :
 - `fgets(buffer, sizeof(buffer), stdin)`
 - `snprintf`
 - `strncpy`

Protection modernes

Exemple corrigé:

```
void safe() {  
    char buf[10];  
    fgets(buf, sizeof(buf), stdin);  
}
```

Plan

1. Qu'est-ce que la pile ?
2. Qu'est ce qu'une stack frame
3. Appel d'une fonction: ce qui se passe
4. Récursion et risques
5. Introduction au buffer overflow
6. Exemple reel de code vulnerable
7. Principe d'exploitation
8. Protections modernes
- 9. Conclusion**

Conclusion

- La pile est essentielle pour gérer l'**exécution des fonctions**, les **variables locales**, et les **paramètres**.
- Les fonctions créent un **frame**, qui disparaît après le retour.
- La compréhension de la pile est fondamentale pour :
 - **Comprendre l'assembleur**
 - La **récurtivité**
 - La **gestion de la mémoire**
 - La **sécurité (buffer overflow)**
 - **Debugging et optimisation**

